哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

机器学习实验一

2026年春

目录

1 实验目标

2 Python基础

3 知识点回顾

4 实验任务说明

5 提交方法和评分标准

一、实验目标

- 掌握Windows系统下Anaconda的安装；
- 掌握Jupyter Notebook的使用方法；
- 熟悉Numpy和Pandas的常见分析操作；
- 熟悉sklearn库内机器学习处理方法；
- 理解决策树的构建过程、信息增益的计算以及分类预测过程；
- 掌握决策树的可视化方法；
- 能对比自实现ID3与sklearn决策树的性能差异。

二、Python基础

（1）Python机器学习的首选工具

（2） Python的集成开发环境——Anaconda

（3） Python编程：使用Jupyter

（4） Python第三方包的引用

（5） Numpy库介绍和使用示例

（6） Pandas库介绍和使用示例

（7） Numpy和Pandas的综合应用

（8） Sklearn库相关介绍

二、Python基础

（1）Python机器学习的首选工具

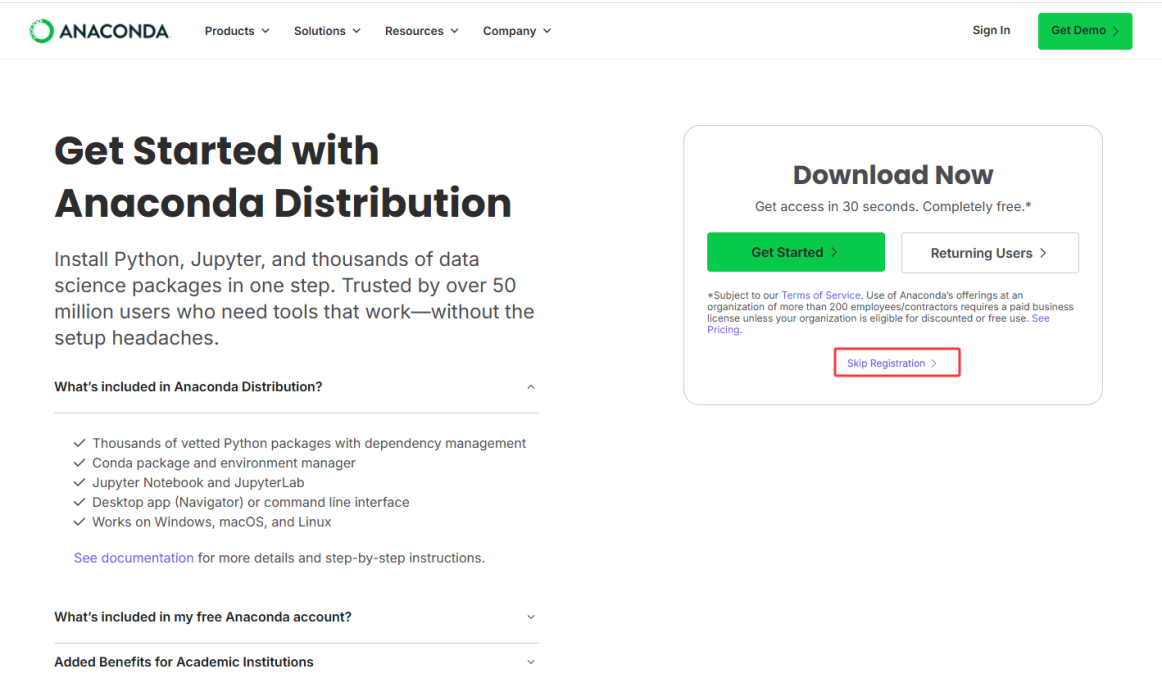
- **Python是一门解释型、面向对象的高级编程语言；**
- **Python是开源免费的、支持交互式、可跨平台移植的脚本语言；**
- **机器学习领域应用广泛：**
  - 简明易用，严谨专业；
  - 良好的开发社区生态；
  - 丰富的第三方程序包（NumPy、Pandas、SciPy、Matplotlib、Scikit-learn等）
  - Scikit-learn：基于 NumPy/SciPy，封装了决策树、SVM、随机森林等常用算法，调用简单，功能强大。

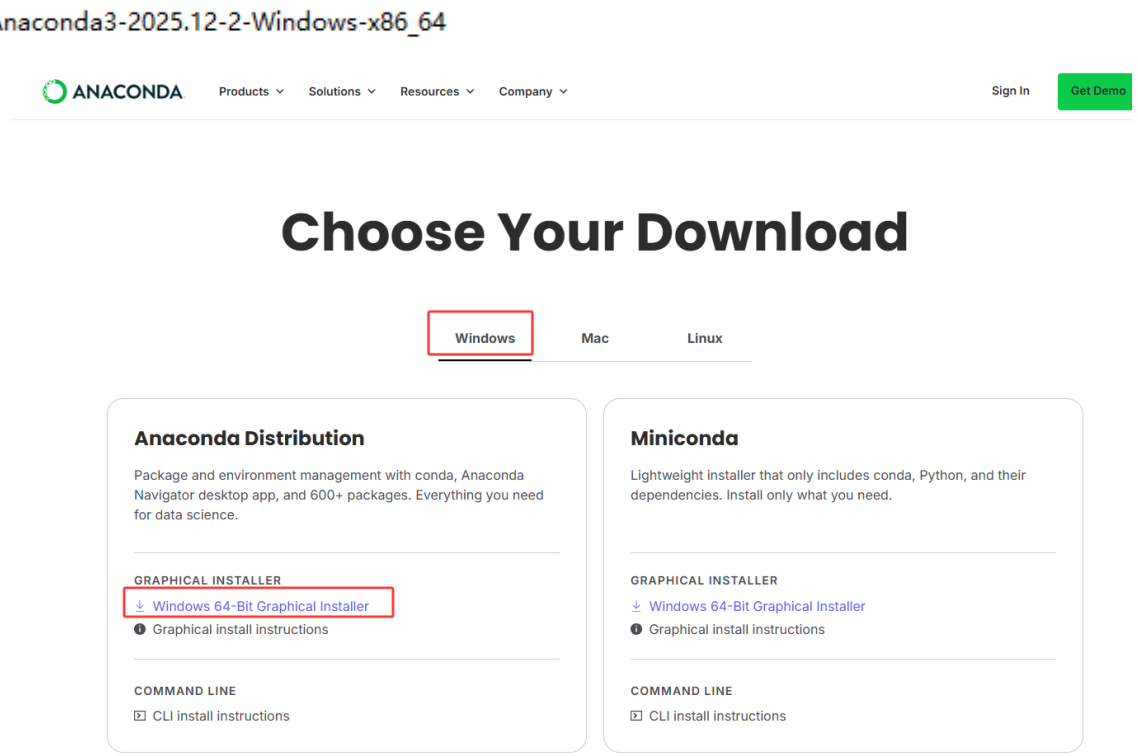
二、Python基础

（2）Python的集成开发环境：Anaconda

◆ **Anaconda简介：是一款兼容Linux、Windows和MacOS环境，支持Python2系列和Python3系列的，且可方便快捷完成机器学习和数据科学任务的开源IDE。Anaconda的官方下载地址为：**

https://www.anaconda.com/download





二、Python基础

（2）Python的集成开发环境：Anaconda

- **学生自己带电脑的，可自行下载安装，另外T2实验室里都有安装好Anaconda环境。**
- **Anaconda安装成功：**

① Anaconda Navigator 桌面图形用户界面（GUI）

② Anaconda Prompt 默认命令提示符工具

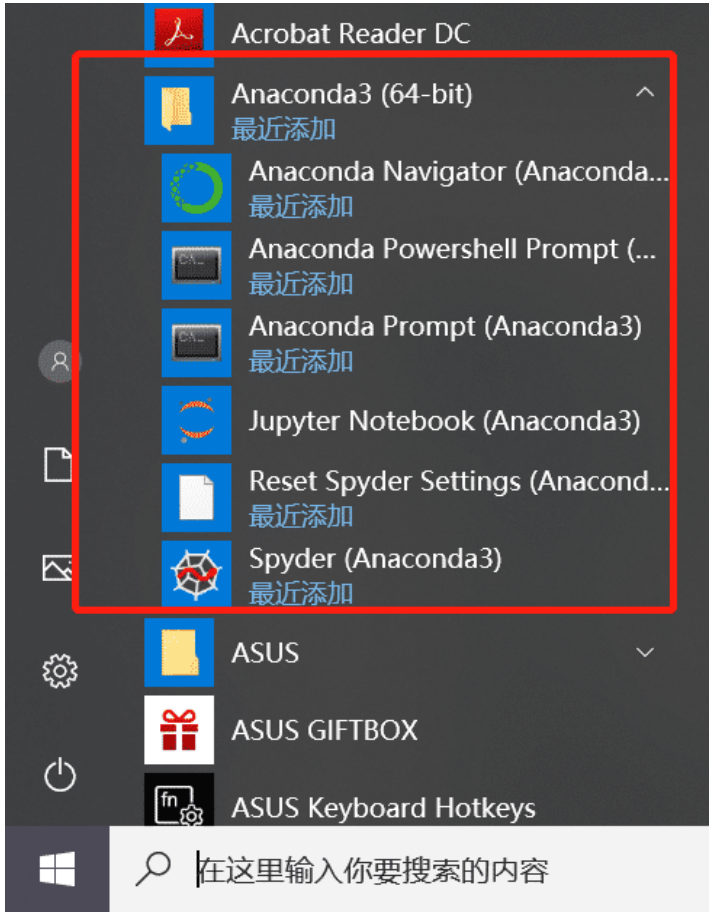
③ **Jupyter Notebook** 交互式笔记本，支持运行 40 多种编程语言

④ Spyder Python 集成开发环境

安装 Anaconda（包含python环境+Jupyter编辑器）

等同

单独安装 Python 环境 +其他 IDE(Pycharm / VScode)

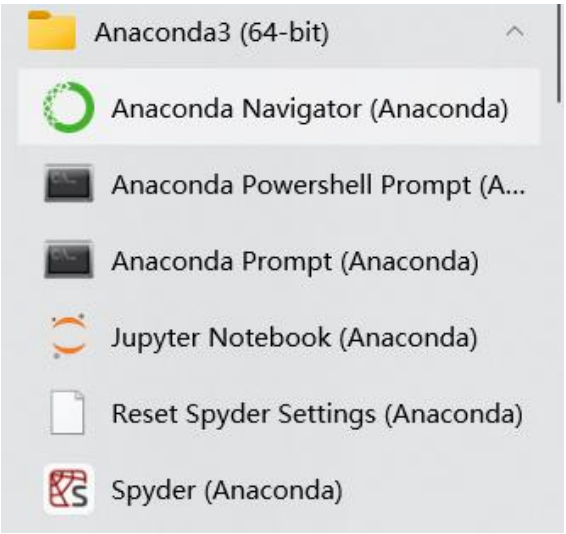


二、Python基础

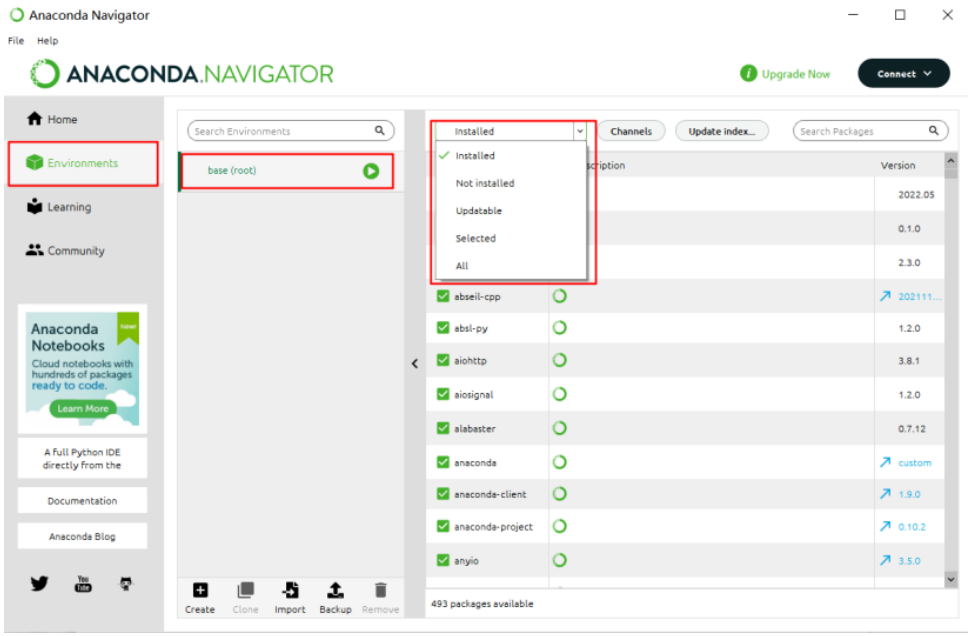
（2）Python的集成开发环境：Anaconda

◆ 管理包

- 依次单击“开始”→ “Anaconda3”选项，单击打开Anaconda Navigator 按钮。



快捷方式

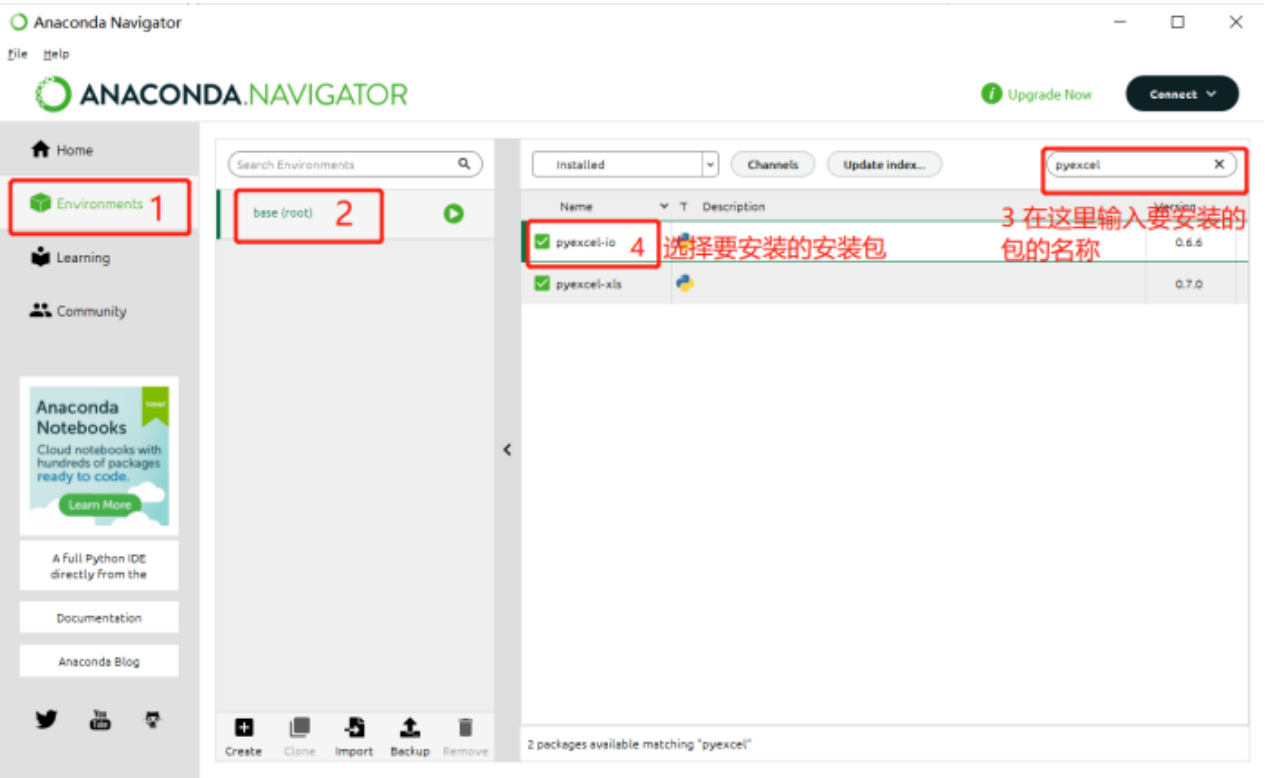


列出已安装的包

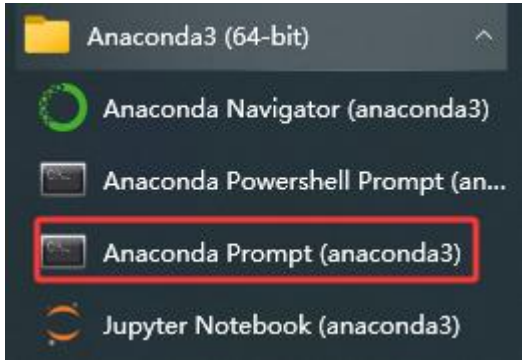
二、Python基础

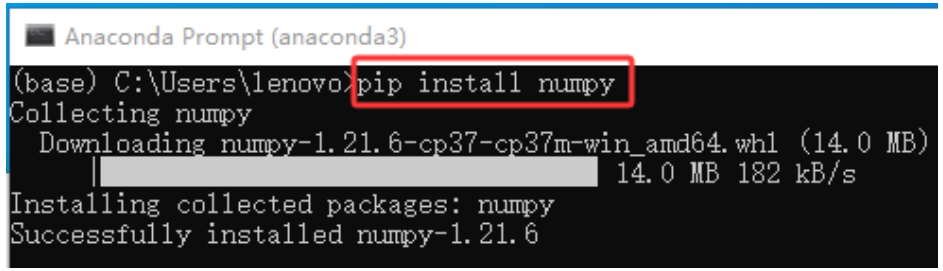
（2）Python的集成开发环境：Anaconda

◆ **管理包**



方法一：安装或者更新包





方法二：打开Anaconda Prompt终端，使用 pip install xxx 命令安装。

二、Python基础

（3）Python编程：使用Jupyter

◆ 认识Jupyter

概念：Jupyter是一个集Python编程语言以及其他编程语言的、交互式集成开发环境，可以将代码执行、文本、数学、情节和富媒体组合到一个文档中。

特点：写代码、看结果、写说明都在一起，Jupyter非常方便。

应用：Jupyter一般用于数据清理和转换、数值模拟、统计建模和机器学习等。

jupyter

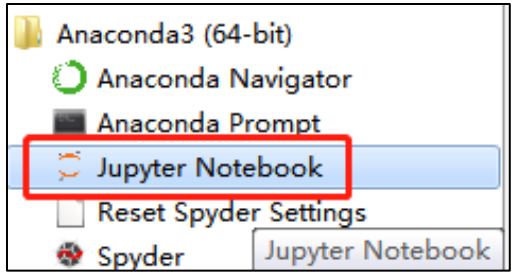
Notebook

二、Python基础

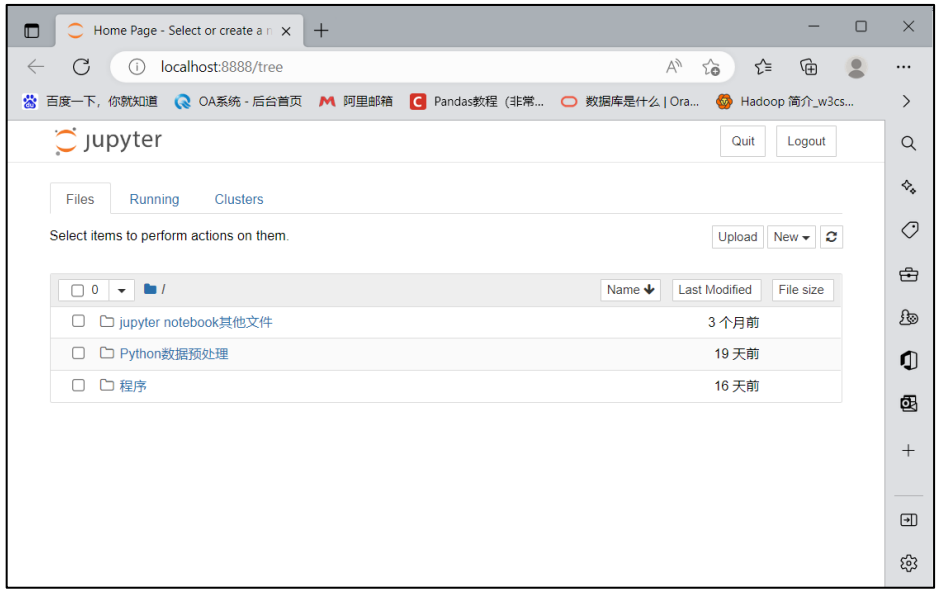
（3）Python编程：使用Jupyter

◆ 启动Jupyter

依次单击“开始”→ Anaconda3 → Jupyter Notebook 按钮，启动Jupyter Notebook应用程序，如下图所示。



启动Jupyter Notebook后，会自动跳转到默认浏览器，浏览器显示结果如下图所示。

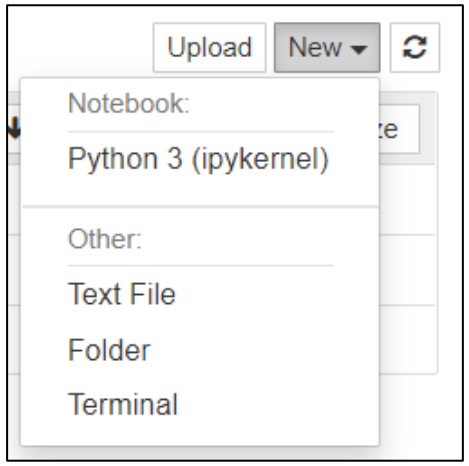


二、Python基础

（3）Python编程：使用Jupyter

◆ 创建新文件

通过单击Files界面中的New按钮创建Jupyter Notebook文件，New下拉菜单如图所示。



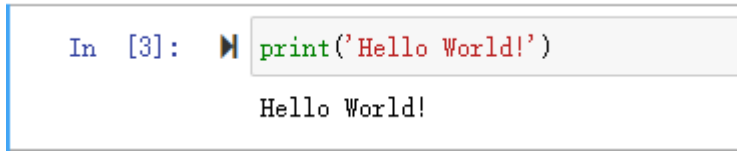
- 此时切换回仪表板，将看到列表中添加了新文件Untitled.ipynb
- .ipynb 文件？**Jupyter Notebook的文件扩展名**

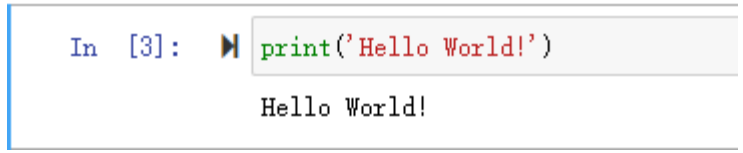
二、Python基础

（3）Python编程：使用Jupyter

◆ 认识界面

- 代码填写在单元格 ① 中
- 输入以下代码：print('Hello World!')
- 点击 ②run 按钮运行程序





二、Python基础

（3）Python编程：使用Jupyter

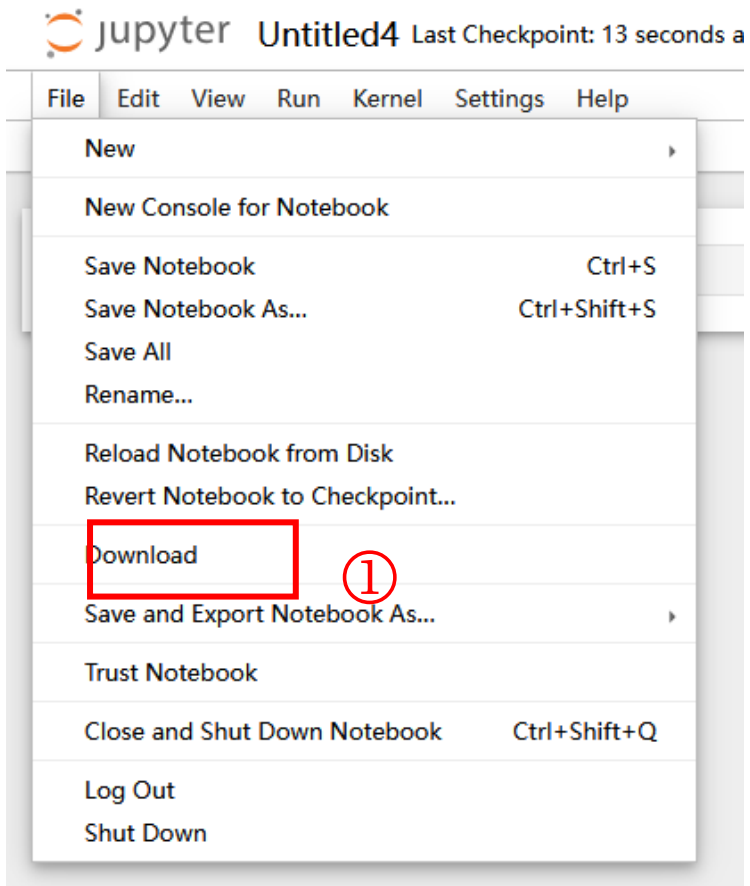
◆ 导入文件

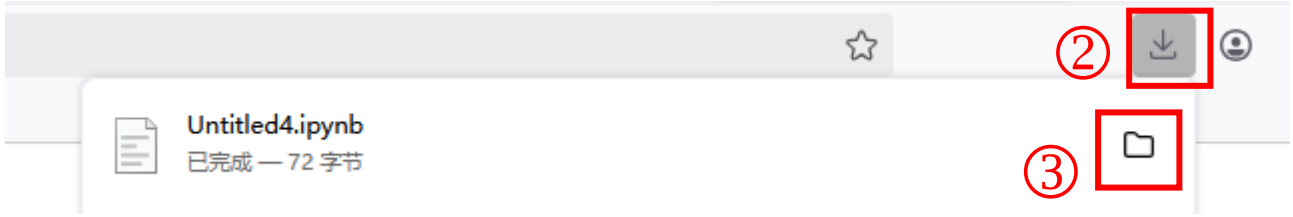
- ① Upload文件
- 点击 ②上传 按钮，上传文件

二、Python基础

（3）Python编程：使用Jupyter

◆ **下载文件**





- ① Download
- 点击 ②下载 按钮
- 点击 ③打开文件 所在的目录

二、Python基础

（3）Python编程：使用Jupyter

◆ **运行结果展示**

```
[1]: i=2

[11]: print(i)
2
```

- 程序调试成功后，结果会显示在单元格下方，下载程序文件该部分内容仍然会保存。

二、Python基础

（4）Python第三方包的引用

- **模块（Module）**：扩展名为 .py 的文件，将代码（变量、函数）打包在一起，可重复使用。
- 导入语法：

```
import 模块名                    # 导入整个模块
from 模块名 import 函数名        # 导入指定函数
from 模块名 import 函数1, 函数2, ... # 导入多个函数
from 模块名 import *             # 导入所有函数（不推荐）
import 模块名 as 别名            # 导入并起别名
```

- 示例：

```
import numpy as np        # 常用别名 np
import pandas as pd       # 常用别名 pd
from sklearn.tree import DecisionTreeClassifier
```

提示：使用别名可以避免不同模块中函数名冲突，也方便书写。

二、Python基础

（5）Numpy库介绍

- Numpy（Numerical Python的简称）是高性能科学计算和数据分析的基础包;
- Numpy 最重要的一个特点是其 N 维数组对象 **ndarray**，它是一系列同类型数据的集合，以 0 下标为开始进行集合中元素的索引;
- ndarray对象的内容可以通过**索引或切片**来访问和修改，与 Python 中 list 的切片操作一样。

- 先在Windows系统终端内执行安装命令 pip install numpy
- 然后在python代码中导入numpy库 import numpy as np

二、Python基础

（5）Numpy库介绍

Numpy库的基本操作

- 查看数组属性
- np.size、np.shape、np.ndim、 np.dtype

- 索引
- b= np.arange(5,20,2)、b[5]、b[2:5]

- 切片
- arr_slice = b[2:5]、 arr_slice[1] = 12345、 arr_slice[:]=64

更多学习请参考numpy中文网：https://www.numpy.org.cn/

二、Python基础

（5）Numpy使用示例

- **创建和访问Numpy的一维数组和二维数组**

```
import numpy as np
data=np.array([1,2,3,4,5,6,7,8,9])
print('Numpy的1维数组:\n{0}'.format(data))
print('数据类型:%s'%data.dtype)
print('1维数组中各元素扩大10倍:\n{0}'.format(data*10))
print('访问第2个元素: {0}'.format(data[1]))
data=np.array([[1,3,5,7,9],[2,4,6,8,10]])
print('Numpy的2维数组:\n{0}'.format(data))
print('访问2维数组中第1行第2列元素: {0}'.format(data[0,1]))
print('访问2维数组中第1行第2至4列元素: {0}'.format(data[0,1:4]))
print('访问2维数组中第1行上的所有元素: {0}'.format(data[0,:]))
```

```
Numpy的1维数组:
[1 2 3 4 5 6 7 8 9]
数据类型:int32
1维数组中各元素扩大10倍:
[10 20 30 40 50 60 70 80 90]
访问第2个元素: 2
Numpy的2维数组:
[[ 1  3  5  7  9]
 [ 2  4  6  8 10]]
访问2维数组中第1行第2列元素: 3
访问2维数组中第1行第2至4列元素: [3 5 7]
访问2维数组中第1行上的所有元素: [1 3 5 7 9]
```

- **Numpy数组的数据来源：Python列表**

```
data=[[1,2,3,4,5,6,7,8,9],['A','B','C','D','E','F','G','H','I']]
print('data是Python的列表(list):\n{0}'.format(data))
MyArray1=np.array(data)
print('MyArray1是Numpy的N维数组:\n%s\nMyarray1的形状:%s'%(MyArray1,MyArray1.shape))
```

```
data是Python的列表(list):
[[1, 2, 3, 4, 5, 6, 7, 8, 9], ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I']]
MyArray1是Numpy的N维数组:
[['1' '2' '3' '4' '5' '6' '7' '8' '9']
 ['A' 'B' 'C' 'D' 'E' 'F' 'G' 'H' 'I']]
Myarray1的形状:(2, 9)
```

代码详细注释请查看老师下发的.ipynb文件

lab1-numpy使用示例.ipynb

二、Python基础

（5）Numpy使用示例——Numpy的计算功能

- **数组计算**

```
MyArray2=np.arange(10)
print('MyArray2:\n{0}'.format(MyArray2))
print('MyArray2的基本描述统计量:\n均值: %f, 标准差: %f, 总和: %f, 最大值: %f'%(MyArray2.mean(),MyArray2.
print('MyArray2的累计和: {0}'.format(MyArray2.cumsum()))
print('MyArray2开平方:{0}'.format(np.sqrt(MyArray2)))
np.random.seed(123)
MyArray3=np.random.randn(10)
print('MyArray3:\n{0}'.format(MyArray3))
print('MyArray3排序结果:\n{0}'.format(np.sort(MyArray3)))
print('MyArray3四舍五入到最近整数:\n{0}'.format(np.rint(MyArray3)))
print('MyArray3各元素的正负号:{0}'.format(np.sign(MyArray3)))
print('MyArray3各元素非负数的显示"正", 负数显示"负":\n{0}'.format(np.where(MyArray3>0,'正','负')))
print('MyArray2+MyArray3的结果:\n{0}'.format(MyArray2+MyArray3))
```

```
 -2.42667924 -0.42891263  1.26593626 -0.8667404 ]
MyArray3排序结果:
[-2.42667924 -1.50629471 -1.0856306  -0.8667404  -0.57860025 -0.42891263
  0.2829785   0.99734545  1.26593626  1.65143654]
MyArray3四舍五入到最近整数:
[-1.  1.  0. -2. -1.  2. -2. -0.  1. -1.]
MyArray3各元素的正负号:[-1.  1.  1. -1. -1.  1. -1. -1.  1. -1.]
MyArray3各元素非负数的显示"正", 负数显示"负":
['负' '正' '正' '负' '负' '正' '负' '负' '正' '负']
MyArray2+MyArray3的结果:
[-1.0856306   1.99734545  2.2829785   1.49370529  3.42139975  6.65143654
  3.57332076  6.57108737  9.26593626  8.1332596 ]
```

- **创建矩阵和矩阵乘法**

```
np.random.seed(123)
X=np.floor(np.random.normal(5,1,(2,5)))
Y=np.eye(5)
print('X:\n{0}'.format(X))
print('Y:\n{0}'.format(Y))
print('X和Y的点积: \n{0}'.format(np.dot(X,Y)))
```

```
. 4.]
 [6. 2. 4. 6. 4.]]
Y:
[[1. 0. 0. 0. 0.]
 [0. 1. 0. 0. 0.]
 [0. 0. 1. 0. 0.]
 [0. 0. 0. 1. 0.]
 [0. 0. 0. 0. 1.]]
X和Y的点积:
[[3. 5. 5. 3. 4.]
 [6. 2. 4. 6. 4.]]
```

- **矩阵运算初步**

```
from numpy.linalg import inv,svd,eig,det
X=np.random.randn(5,5)
print(X)
mat=X.T.dot(X)
print(mat)
print('矩阵mat的逆: \n{0}'.format(inv(mat)))
print('矩阵mat的行列式值: \n{0}'.format(det(mat)))
print('矩阵mat的特征值和特征向量: \n{0}'.format(eig(mat)))
print('对矩阵mat做奇异值分解: \n{0}'.format(svd(mat)))
```

二、Python基础

（6）Pandas库介绍

- Pandas是Python第三方库，提供高性能易用数据类型和分析工具
- Pandas中有两大核心数据结构：Series（一维数据） 和 DataFrame（多特征数据，既有行索引，又有列索引）

Series	
index	value
0	12
1	4
2	7
3	9

Series的数据结构为"键值对"的形式

键—>可以重复

- 安装命令 pip install pandas
- 导入pandas库 import pandas as pd

DataFrame			
index	writer	title	price
0	mark	cookbook	23.56
1	barket	HTML5	50.70
2	tom	Python	12.30
3	job	Numpy	28.00

DataFrame
可以进行
行索引
列索引

是Pandas中重要的数据结构

二、Python基础

（6）Pandas库介绍

Pandas库的基本操作

- 查看数据
  df.describe()、df.index 、df.columns 、df.values

- 选取特定列和行的数据
  df[col]、df[[col1,col2]]、df.iloc[[row1, row2], [col1, col2]]

- 增加 df.append( ) 、 删除某一列 del pf[col] 、 修改列的值 df.pop
- 合并文件 df.merge( ) 、处理缺失值 df.dropna( )

- 导入导出文件
  df.read_csv、df.to_csv

更多学习请参考pandas中文网： https://www.pypandas.cn/

二、Python基础

（6）Pandas使用示例

• Pandas的序列和索引

```
import numpy as np
import pandas as pd
from pandas import Series,DataFrame
data=Series([1,2,3,4,5,6,7,8,9],index=['ID1','ID2','ID3','ID4','ID5','ID6','ID7','ID8','ID9'])
print('序列中的值:\n{0}'.format(data.values))
print('序列中的索引:\n{0}'.format(data.index))
print('访问序列的第1和第3上的值:\n{0}'.format(data[[0,2]]))
print('访问序列索引为ID1和ID3上的值:\n{0}'.format(data[['ID1','ID3']]))
print('判断ID1索引是否存在:%s；判断ID10索引是否存在:%s'%('ID1' in data,'ID10' in data))
```

```
序列中的值:
[1 2 3 4 5 6 7 8 9]
序列中的索引:
Index(['ID1', 'ID2', 'ID3', 'ID4', 'ID5', 'ID6', 'ID7', 'ID8', 'ID9'], dtype='object')
访问序列的第1和第3上的值:
ID1    1
ID3    3
dtype: int64
访问序列索引为ID1和ID3上的值:
ID1    1
ID3    3
dtype: int64
判断ID1索引是否存在:True；判断ID10索引是否存在:False
```

• Pandas的数据框

```
import pandas as pd
from pandas import Series,DataFrame
data=pd.read_excel('北京市空气质量数据.xlsx')
print('date的类型：{0}'.format(type(data)))
print('数据框的行索引：{0}'.format(data.index))
print('数据框的列名：{0}'.format(data.columns))
print('访问AQI和PM2.5所有值：\n{0}'.format(data[['AQI','PM2.5']]))
print('访问第2至3行的AQI和PM2.5：\n{0}'.format(data.loc[1:2,['AQI','PM2.5']]))
print('访问索引1至索引2的第2和4列：\n{0}'.format(data.iloc[1:3,[1,3]]))
data.info()
```

```
date的类型：<class 'pandas.core.frame.DataFrame'>
数据框的行索引：RangeIndex(start=0, stop=2155, step=1)
数据框的列名：Index(['日期', 'AQI', '质量等级', 'PM2.5', 'PM10',
访问AQI和PM2.5所有值：
      AQI  PM2.5
0      81     45
1     145    111
2      74     47
3     149    114
4     119     91
...   ...    ...
2150  183    138
2151  175    132
2152   30      7
2153   40     13
2154   73     38
```

• Pandas的数据加工处理

```
import numpy as np
import pandas as pd
from pandas import Series,DataFrame
df1=DataFrame({'key':['a','d','c','a','b','d','c'],'var1':range(7)})
df2=DataFrame({'key':['a','b','c','c'],'var2':[0,1,2,2]})
df=pd.merge(df1,df2,on='key',how='outer')
df.iloc[0,2]=np.NaN
df.iloc[5,1]=np.NaN
print('合并后的数据:\n{0}'.format(df))
df=df.drop_duplicates()
print('删除重复数据行后的数据:\n{0}'.format(df))
print('判断是否为缺失值:\n{0}'.format(df.isnull()))
print('判断是否不为缺失值:\n{0}'.format(df.notnull()))
print('删除缺失值后的数据:\n{0}'.format(df.dropna()))
fill_value=df[['var1','var2']].apply(lambda x:x.mean())
print('以均值替换缺失值:\n{0}'.format(df.fillna(fill_value)))
```

```
合并后的数据:
  key  var1  var2
0   a   0.0   NaN
1   a   3.0   0.0
2   d   1.0   NaN
3   d   5.0   NaN
4   c   2.0   2.0
5   c   NaN   2.0
6   c   6.0   2.0
7   c   6.0   2.0
8   b   4.0   1.0

删除重复数据行后的数据:
  key  var1  var2
0   a   0.0   NaN
1   a   3.0   0.0
2   d   1.0   NaN
3   d   5.0   NaN
4   c   2.0   2.0
5   c   NaN   2.0
6   c   6.0   2.0
8   b   4.0   1.0

判断是否为缺失值:
    key   var1   var2
0  False  False   True
1  False  False  False
2  False  False   True
3  False  False   True
4  False  False  False
5  False   True  False
6  False  False  False
8  False  False  False

判断是否不为缺失值:
    key   var1   var2
0   True   True  False
1   True   True   True
2   True   True  False
3   True   True  False
4   True   True   True
5   True  False   True
6   True   True   True
8   True   True   True

删除缺失值后的数据:
  key  var1  var2
1   a   3.0   0.0
4   c   2.0   2.0
6   c   6.0   2.0
8   b   4.0   1.0

以均值替换缺失值:
  key  var1  var2
0   a   0.0   1.4
1   a   3.0   0.0
2   d   1.0   1.4
3   d   5.0   1.4
4   c   2.0   2.0
5   c   3.0   2.0
6   c   6.0   2.0
8   b   4.0   1.0
```

代码详细注释请查看老师下发的.ipynb文件

lab1-pandas使用示例.ipynb

二、Python基础

（7）Numpy和Pandas的综合应用——空气质量监测数据的预处理和基本分析

- **数据预处理**

```python
import numpy as np
import pandas as pd
from pandas import Series,DataFrame

data=pd.read_excel('北京市空气质量数据.xlsx')
data=data.replace(0,np.NaN)
data['年']=data['日期'].apply(lambda x:x.year)
month=data['日期'].apply(lambda x:x.month)
quarter_month={'1':'一季度','2':'一季度','3':'一季度',
               '4':'二季度','5':'二季度','6':'二季度',
               '7':'三季度','8':'三季度','9':'三季度',
              '10':'四季度','11':'四季度','12':'四季度'}
data['季度']=month.map(lambda x:quarter_month[str(x)])
bins=[0,50,100,150,200,300,1000]
data['等级']=pd.cut(data['AQI'],bins,labels=['一级优','二级良','三级轻度污染','四级中度污染','五级重度污染','六级严重污染'])
print('对AQI的分组结果: \n{0}'.format(data[['日期','AQI','等级','季度']]))
```

```
   4     2014-01-05  119.0  三级轻度污染   一季度
  ...           ...    ...       ...  ...
  2150 2019-11-22  183.0  四级中度污染   四季度
  2151 2019-11-23  175.0  四级中度污染   四季度
  2152 2019-11-24   30.0     一级优   四季度
  2153 2019-11-25   40.0     一级优   四季度
  2154 2019-11-26   73.0     二级良   四季度

  [2155 rows x 4 columns]
```

- **基本分析**

```python
print('各季度AQI和PM2.5的均值:\n{0}'.format(data.loc[:,['AQI','PM2.5']].groupby(data['季度']).mean()))
print('各季度AQI和PM2.5的描述统计量:\n',data.groupby(data['季度'])['AQI','PM2.5'].apply(lambda x:x.describe()))

def top(df,n=10,column='AQI'):
    return df.sort_values(by=column,ascending=False)[:n]
print('空气质量最差的5天:\n',top(data,n=5)[['日期','AQI','PM2.5','等级']])
print('各季度空气质量最差的3天:\n',data.groupby(data['季度']).apply(lambda x:top(x,n=3)[['日期','AQI','PM2.5','等级']]))
print('各季度空气质量情况:\n',pd.crosstab(data['等级'],data['季度'],margins=True,margins_name='总计',normalize=False))
```

```
各季度AQI和PM2.5的均值:
          AQI      PM2.5
季度
一季度  109.327778  77.225926
三季度   98.911071  49.528131
二季度  109.369004  55.149723
四季度  109.612403  77.195736
```

```
各季度AQI和PM2.5的描述统计量:
                  AQI        PM2.5
季度
一季度 count  540.000000  540.000000
    mean   109.327778   77.225926
    std     80.405408   73.133857
    min     26.000000    4.000000
    25%     48.000000   24.000000
    50%     80.000000   53.000000
    75%    145.000000  109.250000
    max    470.000000  454.000000
三季度 count  551.000000  551.000000
    mean    98.911071   49.528131
    std     45.484516   35.394897
    min     28.000000    3.000000
    25%     60.000000   23.000000
    50%     95.000000   41.000000
```

二、Python基础

（7）Numpy和Pandas的综合应用——空气质量监测数据的预处理和基本分析

- **自拟新的变量**

```
pd.get_dummies(data['等级'])
data.join(pd.get_dummies(data['等级']))
```

	日期	AQI	质量等级	PM2.5	PM10	SO2	CO	NO2	O3	年	季度	等级	一级优	二级良	三级轻度污染	四级中度污染	五级重度污染	六级严重污染
0	2014-01-01	81.0	良	45.0	111.0	28.0	1.5	62.0	52.0	2014	一季度	二级良	0	1	0	0	0	0
1	2014-01-02	145.0	轻度污染	111.0	168.0	69.0	3.4	93.0	14.0	2014	一季度	三级轻度污染	0	0	1	0	0	0
2	2014-01-03	74.0	良	47.0	98.0	29.0	1.3	52.0	56.0	2014	一季度	二级良	0	1	0	0	0	0
3	2014-01-04	149.0	轻度污染	114.0	147.0	40.0	2.8	75.0	14.0	2014	一季度	三级轻度污染	0	0	1	0	0	0
4	2014-01-05	119.0	轻度污染	91.0	117.0	36.0	2.3	67.0	44.0	2014	一季度	三级轻度污染	0	0	1	0	0	0

代码详细注释请查看老师下发的.ipynb文件

lab1-Numpy和Pandas的综合应用.ipynb

- **数据集抽取**

```
np.random.seed(123)
sampler=np.random.randint(0,len(data),10)
print(sampler)
sampler=np.random.permutation(len(data))[:10]
print(sampler)

data.take(sampler)
data.loc[data['质量等级']=='优',:]
```

```
[1346 1122 1766 2154 1147 1593 1761   96   47   73]
[1883  326   43 1627 1750 1440  993 1469 1892  865]
```

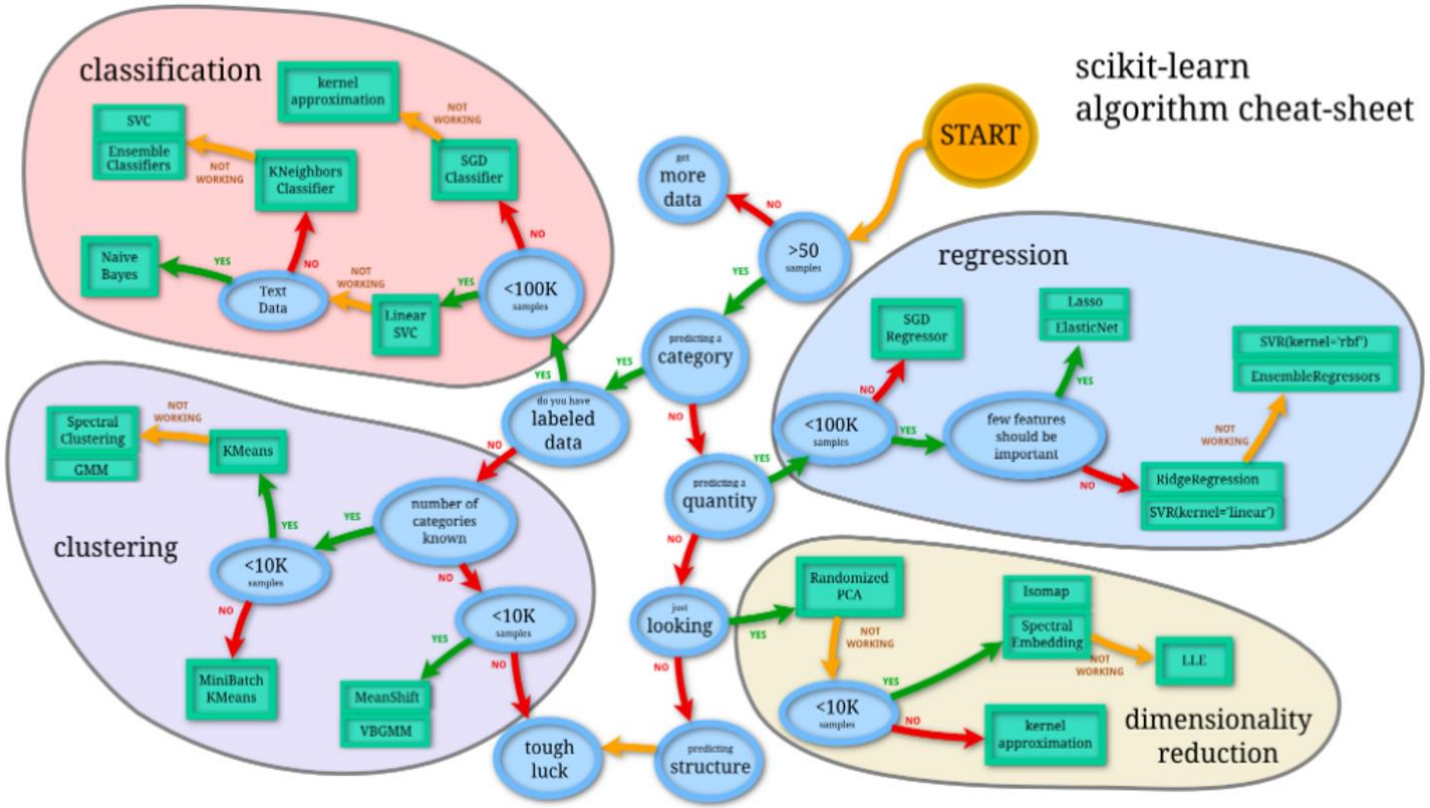
	日期	AQI	质量等级	PM2.5	PM10	SO2	CO	NO2	O3	年	季度	等级
7	2014-01-08	27.0	优	15.0	25.0	13.0	0.5	21.0	53.0	2014	一季度	一级优
8	2014-01-09	46.0	优	27.0	46.0	19.0	0.8	35.0	53.0	2014	一季度	一级优
11	2014-01-12	47.0	优	27.0	47.0	27.0	0.7	39.0	59.0	2014	一季度	一级优
19	2014-01-20	35.0	优	8.0	35.0	6.0	0.3	15.0	65.0	2014	一季度	一级优
20	2014-01-21	26.0	优	18.0	25.0	27.0	0.7	34.0	50.0	2014	一季度	一级优
...	...	...	...	...	...	...	...	...	...	...	...	...
2122	2019-10-25	30.0	优	8.0	20.0	2.0	0.4	24.0	55.0	2019	四季度	一级优
2131	2019-11-03	48.0	优	33.0	48.0	3.0	0.6	34.0	33.0	2019	四季度	一级优
2135	2019-11-07	47.0	优	24.0	47.0	3.0	0.5	37.0	44.0	2019	四季度	一级优
2152	2019-11-24	30.0	优	7.0	30.0	3.0	0.2	11.0	58.0	2019	四季度	一级优
2153	2019-11-25	40.0	优	13.0	30.0	3.0	0.4	32.0	29.0	2019	四季度	一级优

387 rows × 12 columns

二、Python基础

（8）Sklearn库介绍

- **Sklearn** (scikit-learn) 是基于 Python 语言的机器学习工具。SKlearn包含了许多最常见的机器学习算法，例如分类、回归、聚类、数据降维、数据预处理等。



二、Python基础

（8）Sklearn库介绍

- **官方网址：** https://scikit-learn.org
- 功能：只需要调用几行sklearn 库里的API即可实现一个复杂的机器学习算法。
- 安装命令 pip install scikit-learn
- SKlearn的一般步骤：

获取数据，创建数据集 → 数据预处理 → 数据集划分 → 模型定义与训练 → 模型预测与评估 → 模型优化与调参

二、Python基础

（8）Sklearn示例——使用决策树模型实现鸢尾花分类

◆ **鸢尾花数据集**

- 总共包含150行数据
- 每一行数据由 4 个特征值及1个目标值组成。
- 4 个特征值分别为：

  花萼长度、花萼宽度、花瓣长度、花瓣宽度
- 目标值为三种不同类别的鸢尾花，分别为:

  Iris Setosa

  Iris Versicolour

  Iris Virginica

```
5.1,3.5,1.4,0.2,Iris-setosa
4.9,3.0,1.4,0.2,Iris-setosa
4.7,3.2,1.3,0.2,Iris-setosa
4.6,3.1,1.5,0.2,Iris-setosa
5.0,3.6,1.4,0.2,Iris-setosa
5.4,3.9,1.7,0.4,Iris-setosa
4.6,3.4,1.4,0.3,Iris-setosa
5.0,3.4,1.5,0.2,Iris-setosa
4.4,2.9,1.4,0.2,Iris-setosa
4.9,3.1,1.5,0.1,Iris-setosa
5.4,3.7,1.5,0.2,Iris-setosa
```

二、Python基础

（8）Sklearn示例——使用决策树模型实现鸢尾花分类

步骤1：导入必要库

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
```

步骤2：加载数据并查看格式

```
iris = load_iris()
X = iris.data          # 特征矩阵 (150, 4)
y = iris.target        # 标签向量 (150,)
feature_names = iris.feature_names
class_names = iris.target_names

print("特征维度:", X.shape)
print("前5个样本:\n", X[:5])
print("标签示例:", y[:5])
```

步骤3：划分训练集和测试集

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

步骤4：训练决策树模型（使用信息熵作为划分标准）

```
# criterion='entropy' 对应 ID3 的信息增益
clf = DecisionTreeClassifier(criterion='entropy', max_depth=3, random_state=42)
clf.fit(X_train, y_train)
```

步骤5：预测评估

```
y_pred = clf.predict(X_test)
acc = accuracy_score(y_test, y_pred)
print(f"测试集准确率: {acc:.2f}")
```

```
测试集准确率: 0.98
```

二、Python基础

（8）Sklearn示例——使用决策树模型实现鸢尾花分类

步骤6：可视化决策树

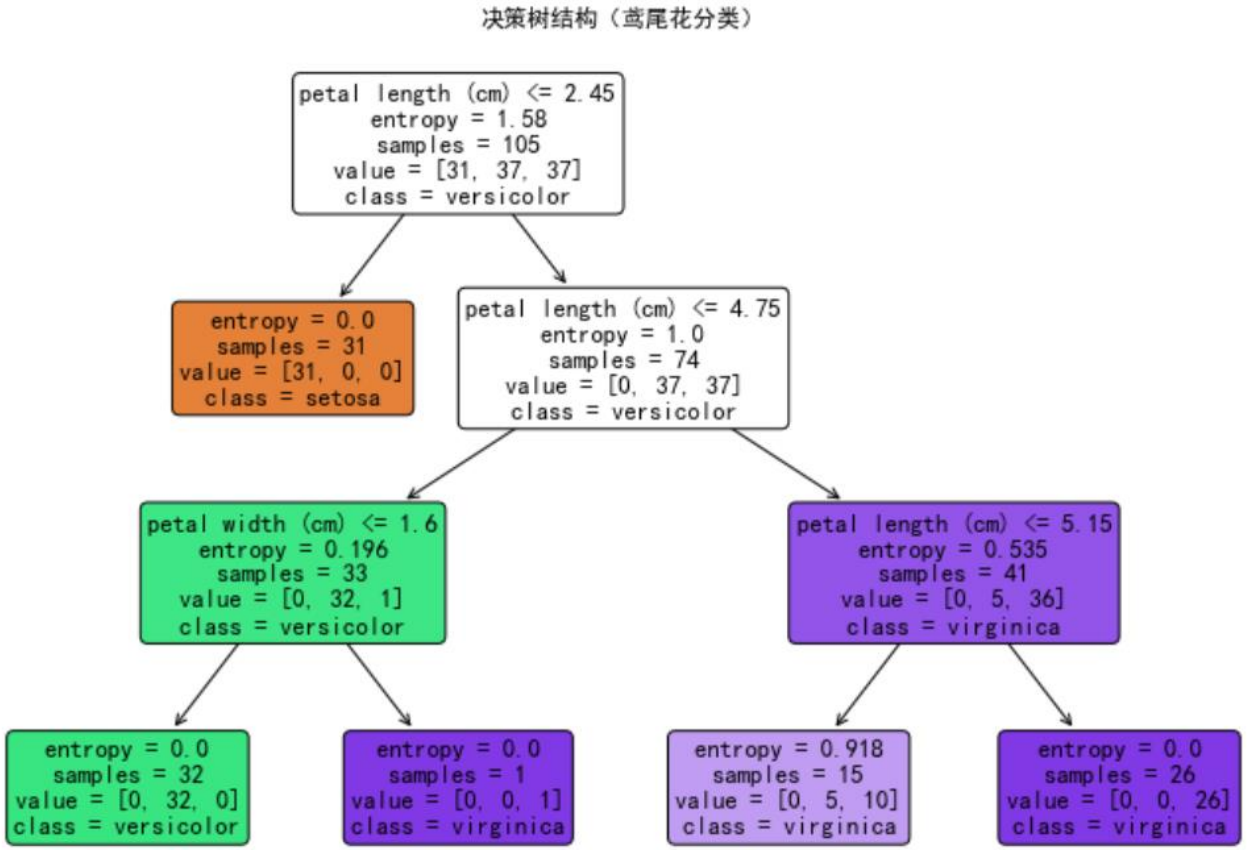
```
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

plt.rcParams['font.sans-serif']=['SimHei']  #解决中文显示乱码问题
plt.rcParams['axes.unicode_minus']=False

plt.figure(figsize=(12, 8))
plot_tree(clf, feature_names=feature_names, class_names=class_names,
          filled=True, rounded=True)
plt.title("决策树结构（鸢尾花分类）")
plt.show()
```

代码详细注释请查看老师下发的.ipynb文件

lab1-sklearn使用示例



三、知识点回顾

数据集划分

银行借贷数据集test
银行借贷数据集train

- 训练集（Training Dataset）是用来训练模型使用的。
- 验证集（Validation Dataset）来看看模型在新数据（验证集和测试集是不同的数据）上的表现如何。
- 测试集（Test Dataset）来做最终的评估。

说明：

1、验证集不像训练集和测试集，它是非必需的。如果不需要调整超参数，就可以不使用验证集，直接用测试集来评估效果。

2、验证集评估出来的效果并非模型的最终效果，主要是用来调整超参数的，模型最终效果以测试集的评估结果为准。

三、知识点回顾

数据集划分

- **基本准则：保持训练集、验证集、测试集之间的互斥性。**
- **参考原则：**

1、对于小规模样本集（几万量级），常用的分配比例是 60% 训练集、20% 验证集、20% 测试集。

2、对于大规模样本集（百万级以上），只要验证集和测试集的数量足够即可，例如有 100w 条数据，那么留 1w 验证集，1w 测试集即可。1000w 的数据，同样留 1w 验证集和 1w 测试集。

3、超参数越少，或者超参数很容易调整，那么可以减少验证集的比例，更多的分配给训练集。

银行借贷数据集test
银行借贷数据集train

三、知识点回顾

数据集划分方法

◆ **划分方法：** **K折交叉验证**（一种动态验证的方式，这种方式可以降低数据划分带来的影响）

以10折交叉验证为例，具体步骤如下：

1、将数据集分为训练集和测试集，**将测试集放在一边**；

2、将训练集平均分成不相交的10个子集；

3、每一次挑选其中的1份作为验证集，其余的9份作为训练集进行模型训练，得到模型以及评价指标；

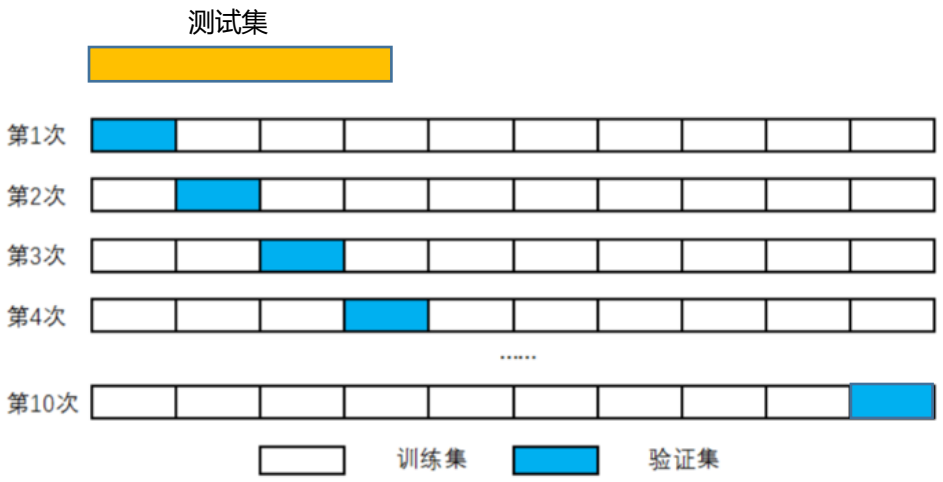
4、重复第3步10次，通过 10 次训练后，得到了 10个不同的模型；

5、将10个模型的评价指标取平均值，作为交叉验证的评估指标；

6、使用不同的超参数，重复以上2-5步，根据最好的交叉验证评估指标，挑选出最优的超参数;

7、使用最优的超参数，将数据全部作为训练集重新训练模型；

8、最后使用测试集测试**评估模型**，计算**评价指标**。



三、知识点回顾

评价模型

◆ 在二分类任务中，各指标的计算基础都来自于对正负样本的分类结果，用混淆矩阵表示为：

真实情况	预测结果	
	正例	反例
正例	TP	FN
反例	FP	TN

◆ 根据混淆矩阵，得出四个评价指标，分别为：

指标	公式	含义
准确率 (Accuracy)	(TP+TN)/(TP+TN+FP+FN)	所有样本中预测正确的比例
精确率 (Precision)	TP/(TP+FP)	预测为正的样本中实际为正的比例
召回率 (Recall)	TP/(TP+FN)	实际为正的样本中被正确预测的比例
F1-score	2×Precision×Recall/(Precision+Recall)	精确率与召回率的调和平均

◆ 提示：正负样本平衡时可用准确率指标，不平衡时用F1-score评价指标。

三、知识点回顾

欠拟合、适度拟合、过拟合

◆ **欠拟合**：

定义：训练集和测试集上的准确率都不高，且相差不大，这种情况称为欠拟合（Under-Fitting）。如：一个为80%，另一个为82%。

解决办法：添加其他特征项，模型出现欠拟合的时候是因为特征项不够导致的，可以添加其他特征项来很好地解决。

◆ **适度拟合**：

训练集和测试集的准确率都很高，且相差不大，这种情况称为适度拟合，这是我们想要的结果。如：一个为99%，另一个为98%。

◆ **过拟合**：

定义：训练集准确率 远远大于 测试集准确率，这种情况称为过拟合（Over-Fitting）。如：一个为99%，另一个为88%。

解决办法：正则化、随机失活、逐层归一化、提前终止、Bagging。

更多学习 https://blog.csdn.net/weixin_39852647/article/details/111095814

三、知识点回顾

ID3 算法步骤

输入：训练数据集D，特征集A

输出：决策树T

Step1：若D中所有实例属于同一类C_K，则T为单结点树，并将类C_K作为该节点的类标记，返回T；

Step2：若A=∅，则T为单结点树，并将D中实例数最大的类C_K作为该节点的类标记，返回T；

Step3：否则，计算A中每个特征对D的 **信息增益** ，选择 **信息增益** 最大的特征；

Step4：如果A_g的 **信息增益** 小于阈值ε，则T为单节点树，并将D中实例数最大的类C_K作为该节点的类标记，返回T

Step5：否则，对A_g的每一种可能值a_i，依A_g=a_i 将D分割为若干非空子集 D_i，将D_i中实例数最大的类作为标记，构建子结点，由结点及其子树构成树T，返回T；

Step6：对第i个子节点，以D_i为训练集，以 $A-\{A_g\}$为特征集合，递归调用**Step1~step5**，得到子树T_i，返回T_i。

三、知识点回顾

熵与信息增益

◆ 熵 $Entropy(S) = \sum_{i=1}^{c} -p_i \log_2 p_i$

```python
import math
def calc_ent(data):
    """
    参数: data : list of list，每行是一个样本，最后一列为类别标签
    返回: 熵值 (float)
    """
    # 处理空数据集，避免除零错误
    if not data:
        return 0.0
    data_length = len(data)   # 样本总数
    label_count = {}     # 统计各类别出现次数
    for sample in data:
        label = sample[-1]   # 最后一列是类别
        label_count[label] = label_count.get(label, 0) + 1

    ent = 0.0
    for count in label_count.values():
        p = count / data_length
        ent -= p * math.log2(p)
    return ent
```

◆ 条件熵 $\sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$

```python
def cond_ent(data, axis=0):
    """ 计算条件熵
    参数: data : 数据集（list of list）
    axis : 特征所在的列索引（0-based）
    返回: 条件熵值 (float)
    """
    data_length = len(data)
    feature_sets = {}            # 按特征值分组
    for i in range(data_length):
        feature = data[i][axis]     # 取特征值
        if feature not in feature_sets:
            feature_sets[feature] = []
        feature_sets[feature].append(data[i])

    cond_ent = 0.0
    for value, subset in feature_sets.items():
        weight = len(subset) / data_length       # 权重
        cond_ent += weight * calc_ent(subset) # 加权求和
    return cond_ent
```

◆ 信息增益 $Gain(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$

```python
def info_gain(ent, cond_ent):
    """
    计算信息增益 gain = H(D) - H(D|A)
    """
    return ent - cond_ent
```

三、知识点回顾

ID3算法完整构建流程（伪代码）

```
def build_tree(data, features):
        如果data中样本全属于同一类别 -> 返回该类别叶子节点
        如果features为空 -> 返回data中出现最多的类别
        选择信息增益最大的特征A
        创建根节点，特征 = A
        对于A的每个取值v:
                data_v = data中特征A == v的子集
                如果data_v为空 -> 挂一个叶子节点，类别为父节点多数类
                否则 -> 递归构建子树 build_tree(data_v, features\{A})
        返回根节点
```

四、实验任务说明

◆ **背景**：银行借贷是基于分析历史按时还款、逾期或不还的用户群体的各自特征建立模型，未来借款用户只要符合符合借款要求，就给予借贷，如果不符合，则拒绝。

◆ **任务一**：调用**Sklearn库分类器**，创建一个决策树模型，进行银行贷款预测；

◆ **任务二**：使用**Python 自编程**，实现 **ID3 算法**，创建一个决策树模型，进行银行贷款预测。

◆ **数据集**（训练集800条，测试集200条）

银行借贷数据集test
银行借贷数据集train

① name_id：姓名id

② profession： 职业，1-企业工作者，2-个体经营户，3-自由工作者，4-事业单位，5-体力劳动者

③ education： 教育程度，1-博士及以上，2-硕士，3-本科，4-专科，5-高中及以下

④ house_loan： 是否有房贷，1-有，0-没有

⑤ car_loan：是否有车贷，1-有，0-没有

⑥ married： 是否结婚，1-是，0-否

⑦ child：是否有小孩，1-有，0-没有

⑧ revenue：月收入

⑨ **approve**：是否予以贷款，1-贷款，0-不贷款

nameid	profession	education	house_loan	car_loan	married	child	revenue	approve
1	5	1	0	0	1	1	8204	1
2	3	1	1	1	0	0	5674	0
3	2	3	1	0	1	0	10634	1
4	2	2	0	0	0	0	43551	1
5	4	2	0	1	0	1	14065	0

四、实验任务说明

◆ **任务一**：调用**Sklearn库分类器**，创建一个决策树模型，调整各项参数，计算评价指标，绘制出决策树，最后对测试集做预测评估。具体要求如下：

① **数据读取：**使用**pd.read_excel()**函数读取本地 银行借贷数据集train.xls 文件和 银行借贷数据集test.xls 文件；

② **数据预处理：**确定特征列（X）和标签类别列（Y），以及**划分**训练集、验证集（可从训练集中分出20%作为验证集，也可使用 sklearn.model_selection.cross_val_score 函数来划分）；

③ **模型定义与训练**：**记录≥ 3组参数的调参过程**（如参数 max_depth, min_samples_split, criterion等）；且每组参数组合应在**验证集**上记录**准确率、精确率、召回率、F1**，并说明最终选择的参数理由（如选F1最高的组合）；可使用 sklearn.model_selection.GridSearchCV 或手动调参做记录；

④ **绘制决策树**：可使用 sklearn.tree.plot_tree 直接绘图，但图片可能较拥挤，可适当调整 figsize 和 max_depth。（也可考虑安装graphviz第三库后绘图，请自行了解）；**图片保存**为png格式；

⑤ **评估模型**：计算模型在测试集上的准确率、精确率、召回率和F1值。

四、实验任务说明

◆ **任务二**：使用**Python 自编程**，自编程定义**ID3 算法**，创建决策树模型，对测试集做预测评估。具体要求如下：

① **数据读取和数据预处理**，同任务一，以及数据集中revenue列数据要进行**离散化**（测试集与训练集都要做）；

```
re = [0,10000,20000,30000,40000,50000]
df['revenue']=pd.cut(df['revenue'],re,labels=False)
```

② **模型的定义与训练**：必须实现以下**4个函数（带注释）**，来构造多叉决策树；

```
def calc_entropy(data):                    #计算数据集熵
def calc_info_gain(data, feature, label):  #计算某特征的信息增益
def build_tree(data, features):            #递归构建多叉决策树
def predict(tree, sample):                 #对单个样本预测（含未见过值处理）
```

③ **构建树的停止条件**（**必须实现**）

- 当前样本 **全属同一类别** → 返回该类
- **候选特征集为空** → 返回当前多数类
- 当前数据集 **为空** → 返回父节点多数类（需传入父节点统计）

四、实验任务说明

◆ **任务二**：使用**Python 自编程**，自编程定义**ID3 算法**，创建决策树模型，对测试集做预测评估。具体要求如下：

⑤ **未见过属性值的处理（Fallback）**：在 predict 中，若节点特征取值 v 不存在对应子节点，则 **返回该节点子树中最常见的类别**（多数表决）。

⑥ **模型评估**：自定义精确率P、召回率R和F1值指标来评价模型；

⑦ 与 sklearn 模型对比，结合实验数据具体**分析差异原因**（从以下 ≥2 个维度分析）

- 划分准则（信息增益 vs 基尼系数/信息增益比）
- 连续值处理（手动分箱 vs 自动寻优分割点）
- 过拟合程度（无剪枝 vs 可调 max_depth 等参数）
- 缺失值处理能力
- 其他…

四、实验任务说明

- 选做题：在任务二（自编程 ID3）的基础上，加入**预剪枝（Pre-Pruning）**功能，限制决策树的最大深度，并对比不同深度下的模型性能。具体要求如下：

① 修改 build_tree 函数

- 增加参数 max_depth（默认值不限，如 None 或较大数）；
- 增加参数 current_depth（记录当前深度）；
- 当 current_depth >= max_depth 时，停止递归，生成叶子节点（取当前数据集的多数类）；

② 实验对比

- 分别设置 max_depth = 3, 5, 10, None（不限深度）；
- 在测试集上计算每种深度的 **精确率、召回率、F1 值；**
- 观察深度变化对性能的影响；

③ 分析要求

- 说明随着深度增加，模型在训练集和测试集上的表现分别如何变化；
- 指出是否存在过拟合现象。

提示：选做题有10%的加分， 但总分不超过该次实验满分。

五、提交方式与评分标准

实验报告提交至平台 https://grader.hitsz.edu.cn/courses

注意：

- 1、用户名 可用统一身份登录；
- 2、请提交到相应的条目「2026春-机器学习（理论课老师）」课程 - 实验一；
- 3、提交截止时间：5月8号 晚24点（两周时间）；补交按得分80%计算分数；
- 4、文件夹&压缩包命名要求： 学号_姓名_机器学习实验一.zip；
- 5、提交内容：实验报告(.pdf文件)+两个任务的代码(.py/ipynb文件，各自完整实现，重要代码处加注释)，一起打包为zip格式压缩包。实验报告内容和要求请参考模板；

常见问题与避坑指南

- Q1：**revenue是连续值，直接计算信息增益会出错?**

  A：必须离散化，方法见前面说明。
- Q2：**自编程ID3训练时间很长?**

  A：检查递归是否陷入无限循环（如特征值重复导致子集不变），并确保每次移除已用特征。
- Q3：**sklearn决策树可视化中文乱码?**

  A：设置字体 plt.rcParams['font.sans-serif'] = ['SimHei']。
- Q4：**测试集预测时遇到训练时没见过的属性值?**

  A：可采用多数表决返回父节点类别（在predict函数中实现fallback逻辑）。

推荐学习资源

- Scikit-learn决策树官方中文文档(DecisionTreeClassifier参数详解)：http://sklearn.apachecn.org
- ID3算法完整Python实现（含离散化）：https://developer.baidu.com/article/details/2898914
- 周志华《机器学习》第4章（重点阅读4.2节“划分选择”和4.3节“剪枝处理”）：https://blog.csdn.net/weixin_46034116/article/details/124969465
- B站视频（ID3 决策树 机器学习）：https://www.bilibili.com/video/BV1JvpZeyE3Z/?spm_id_from=333.337.search-card.all.click&vd_source=7815a16f8fb15434bb283fcf2410bd66